

SkyvoltRobot MVP

진행 보고서 — Phase 0 실행/검증 및 Phase 1 착수

작성일: 2026 - 06 - 14

1. 프로젝트 개요

SkyvoltRobot은 레일에 매달려 이동하는 EV 충전 로봇이다. 본 저장소는 ROS 2 기반 디지털 트윈으로, (가) ROS 없이 도는 순수 파이썬 알고리즘 코어(위치추정 칼만필터·도킹 속도정책 FSM·다중로봇 스케줄러)와 (나) Gazebo 3D 시뮬레이션 두 갈래로 구성된다. 이번 작업의 목표는 '실제로 실행되게 만들고', 이어서 Phase 1(위치추정 견고화)의 소프트웨어 작업을 완료하는 것이었다.

2. 작업 요약

- 순수 파이썬 단위 테스트 및 도킹 평가 하니스 실행 — 모든 검증 게이트 통과.
- 플릿 관리 REST API 서버(FastAPI)를 띄우고, 재사용 가능한 api - server/ 디렉토리로 분리.
- Gazebo 3D 시뮬레이션을 실제로 구동 — 단일 로봇 및 3대 플릿. 다수의 빌드/모델/렌더 버그 수정.
- Phase 1 소프트웨어 작업 3건 완료 — #2 IMU+엔코더 융합, #3 속성 기반 테스트, #4 Replay 도구.
- Phase 2 완료 — 스케줄러 스트레스 테스트(1000 작업·10 로봇, 데드락 - free), 예약 데드락 타임아웃·복구 정책, 운영자 대시보드(읽기 전용), Gazebo 페루프 3대 로봇 구동.
- Phase 3 핵심 IP 완료 — 리드스크류 기구학, RGB - D 충전구 자세 추정, 비주얼 서보잉(정렬→삽입→도킹), 힘/토크 정합 피드백.
- Phase 4 핵심 IP 완료 — 트랙 하중·기울기 모니터, 래칭 E - stop 체인, Prometheus 플릿 헬스 메트릭(HTTP /metrics + Grafana 보드), OTA 업데이트 FSM.
- 연동 — Gazebo RGB - D 영상 특징 검출 프론트엔드(영상→자세→서보→도킹 페루프).
- 전체 자동화 테스트 149개 전부 통과. (#1 RFID 실측 모델만 실물 장비 필요로 보류.)

3. Phase 0 — 실행 및 검증

3.1 단위 테스트

ROS의 launch_testing 플러그인이 최신 pytest와 충돌해 테스트 수집이 깨지는 문제를 발견, 서드파티 플러그인 자동로드를 비활성화하여 해결했다. 이후 Phase 1에서 속성·IMU·Replay·스트레스·복구·대시보드·매니플레이터·비전·서보·F/T·안전 테스트를 추가하여 현재 총 149개 테스트 전부 통과한다.

3.2 도킹 평가 하니스 (100회 시뮬레이션)

지표	측정값(p99)	합격 기준	판정
도킹 성공률	100 / 100	—	통과
횡방향 오차	약 2.9 mm	< 4 mm	통과
종방향 오차	약 9.2 mm	< 12 mm	통과
소요 시간	약 10.1 s	< 13 s	통과

최종 결과: PASS — 모든 게이트 충족.

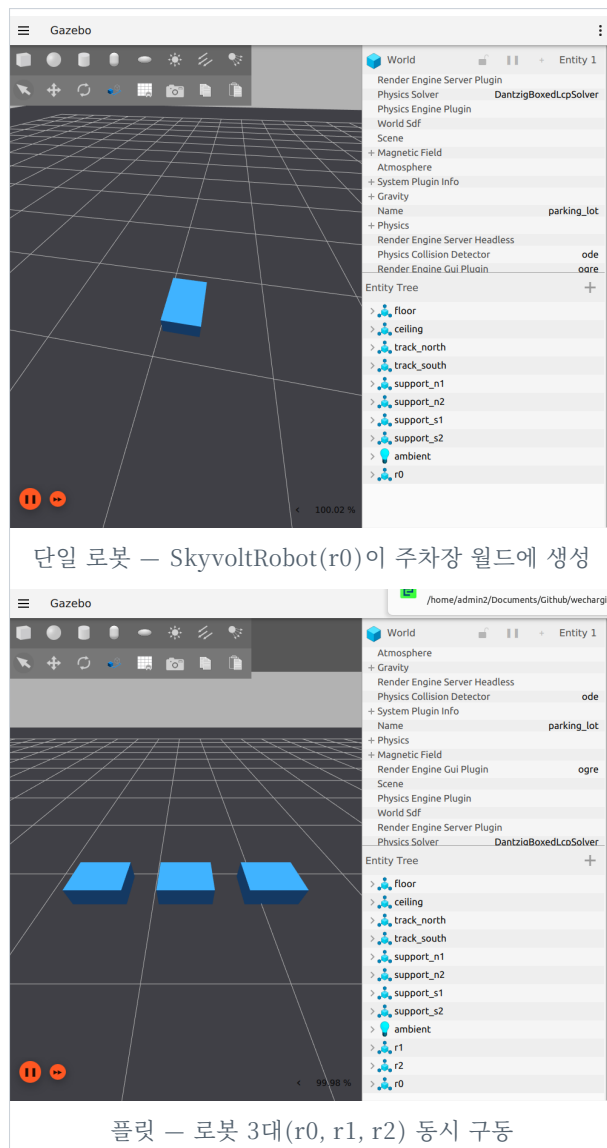
4. 플릿 관리 API 서버

FastAPI 기반 REST 서버를 구동하고(8080 포트), 충전 작업 등록 시 스케줄러가 로봇을 자동 배정하는 것을 확인했다. 서버 실행에 필요한 최소 파일(서버 본체 + 스케줄러.예약 로직)을 추적하여 자체 완결형 api - server/ 디렉토리로 모았다. 이 폴더만 복사하면 어디서든 서버를 띄울 수 있다.

- GET /fleet — 로봇 3대 상태 조회
- POST /jobs — 충전 작업 등록(등록 즉시 자동 배정)
- GET /jobs/{id} — 작업 상태 조회

5. Gazebo 3D 시뮬레이션

ROS 2 Humble + Gazebo Fortress 환경을 설치하고 워크스페이스를 colcon으로 빌드했다. 실행 과정에서 여러 버그(아래 6장)를 수정한 끝에, 단일 로봇과 3대 플릿 시뮬레이션을 실시간(약 100% real - time)으로 구동하는 데 성공했다.



6. 수정한 버그

#	증상	원인 / 조치
1	colcon 빌드 실패	존재하지 않는 meshes 폴더 설치 시도 → CMakeLists에서 제거
2	노드 실행파일 누락	setup.cfg 부재로 bin/에 잘못 설치 → setup.cfg 추가(2개 패키지)

3	launch 파싱 실패	robot_description를 YAML로 오인 → ParameterValue(str) 래핑
4	로봇이 안 뜸	빈 링크에 질량 없어 모델 무시 → 관성 블록 3개 추가(URDF)
5	3D 화면이 검은색	Ogre2가 NVIDIA에서 렌더 실패 → Ogre v1 + NVIDIA EGL 강제
6	run_sim 스크립트 실패	set -u가 ROS setup.bash와 충돌 → -u 제거

이 중 #1~#4는 프로젝트 자체 버그, #5~#6은 환경/스크립트 이슈였다.

7. Phase 1 — 위치추정 견고화

로드맵의 Phase 1 4개 작업 중, 실물 장비가 필요한 #1(RFID 실측 노이즈)을 제외한 소프트웨어 작업 3건을 완료했다.

7.1 #2 — IMU + 휠 엔코더 융합

칼만필터를 확장하되 기존 API와 완전한 하위 호환을 유지했다. (가) IMU 가속도계를 predict()의 제어 입력으로 받아 속도 변화 구간(곡선 진입·접근 감속) 추정을 개선하고, (나) 곡선에서 자이로 각속도로부터 $v = \omega \cdot R$ 속도 추정을 융합해 휠 슬립에 강인하게 했다. ROS 노드는 /imu 토픽을 구독하며, IMU가 끊기면 등속도 모델로 자동 폴백한다.

- 예: 가속 구간에서 IMU 융합이 등속도 모델보다 속도 오차가 작음을 테스트로 입증.
- predict(dt)는 수학적으로 이전과 동일 — 도킹 평가 게이트 PASS 그대로 유지.

7.2 #3 — 속성 기반 fault-injection 테스트

손으로 고른 입력 대신 Hypothesis가 무작위·극단 입력 시퀀스를 생성해, 칼만필터와 FSM이 항상 지켜야 할 불변식을 검증한다(공분산 PSD·유한성·측정이 분산을 키우지 않음, 속도 [0,1.5] 범위·저크 제한·도킹 단계 전진성·터미널 고착 등). 속성 테스트가 처음 작성한 불변식 하나가 과하게 강했음을 즉시 잡아냈고(정상 비상정지를 오판), FSM 코드는 정상이어서 명세를 바로잡았다 — 이것이 속성 기반 테스트의 가치다.

7.3 #4 — Replay 도구 (bag → 오프라인 localizer → 동일 지표)

기록된 센서 스트림을 ROS/Gazebo 없이 오프라인 localizer로 재생해 동일한 위치추정 지표를 재현한다. ROS 독립 JSONL trace 포맷을 정의하고, rosbag2를 읽어 trace로 변환하는 도구(ROS 있을 때)와 순수 파이썬 재생·지표 산출(CI 가능)을 분리했다. localizer가 결정론적이므로 재생 결과는 라이브 추정과 비트 단위로 동일함을 확인했다(504틱 불일치 0).

8. Phase 2 — 다중로봇 (착수)

8.1 스케줄러 스트레스 테스트 (1000 작업 · 10 로봇, 데드락-free)

assign()이 호출당 유휴 로봇 1대에 1작업만 배정하므로, 1000개 작업을 처리하려면 시간을 진행시키는 페루프 시뮬레이션이 필요하다. 작업 배정 → 로봇 이동(busy) → 도착 시 예약 해제·유휴 복귀 → 재배정을 모든 작업 완료까지 반복하며, '대기 작업과 유휴 로봇이 있는데 admit 가능한 경로가 없고 풀어줄 작업도 없는' 진짜 교착을 감지한다. 게이트는 데드락 없음 + 100% 완료 + 잔여 예약 0이다.

지표	결과(1000 작업/10 로봇)
완료율	1000 / 1000 (100%)
데드락	없음
잔여 예약	0 (전부 해제)
처리량	약 1.63 작업/초
판정	PASS

부하 확장성도 확인: 버스트 도착 시 로봇 2대(makespan 134.7 s) → 12대(45.5 s)로 처리량이 선형에 가깝게 확장된다.

8.2 예약 데드락 — 타임아웃 + 복구 정책

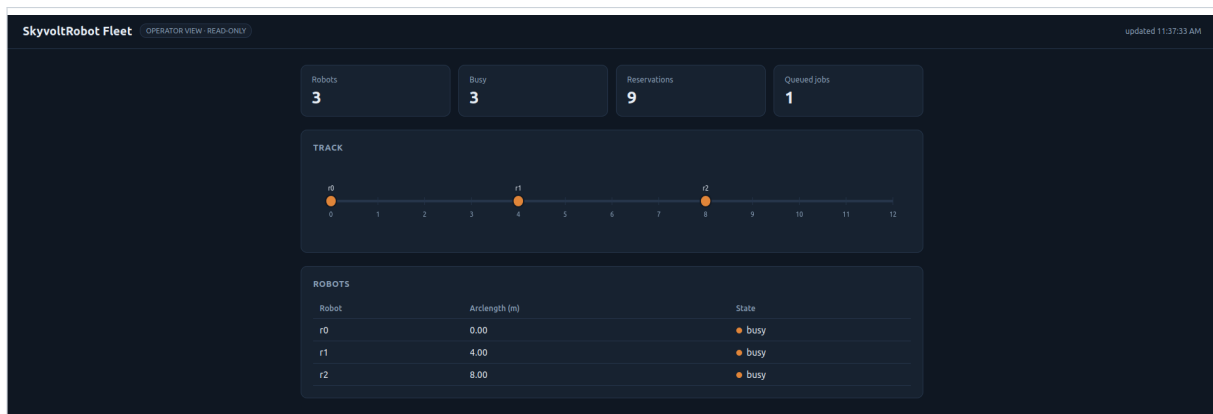
로봇이 멈추면(잼.크래시) 그 예약이 해제되지 않아 트랙을 영구히 막는다. 예약 테이블에 타임아웃 회수(reclaim_expired)를 추가해, 경로 전체가 만료 시한을 넘긴 로봇을 '멈춤'으로 간주하고 예약을 회수한다. 복구 정책은 멈춘 로봇을 서비스에서 제외하고 그 작업을 다시 큐에 넣어 정상 로봇이 마저 처리하게 한다.

멈춘 로봇 시나리오	완료율	데드락	잔여 예약	판정
복구 OFF	0.995	발생	3 (영구 점유)	FAIL
복구 ON	1.000	없음	0	PASS

같은 멈춤 상황에서 복구 정책 하나로 데드락이 100% 완료로 바뀐다.

8.3 운영자 대시보드 (읽기 전용)

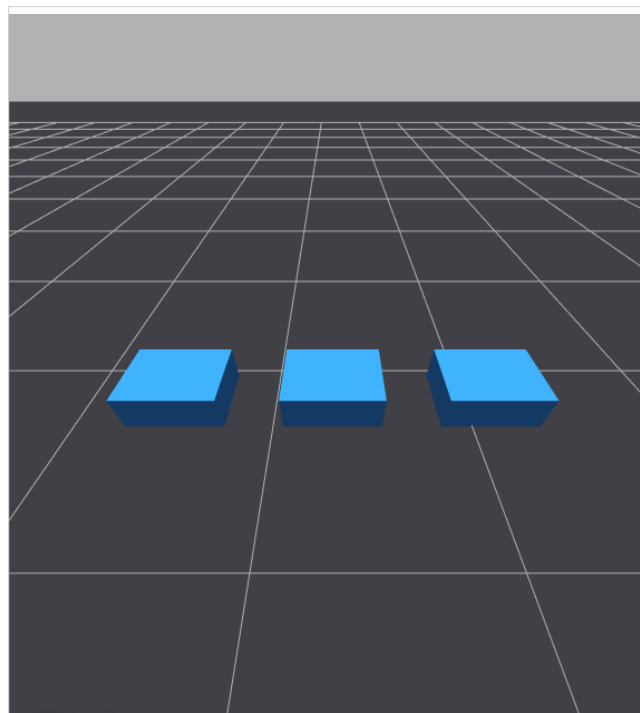
기존 FastAPI 서버에 빌드 도구 없이 동작하는 바닐라 HTML/JS 대시보드를 추가했다. GET / 는 /fleet 을 1초마다 폴링해 트랙 위 로봇 위치(초록=유휴, 주황=작업중), 예약·대기 작업 수를 실시간 표시한다. 작업 등록 컨트롤이 없는 읽기 전용이다.



운영자 대시보드 — 로봇 3대 작업중, 트랙 위 위치 표시

8.4 페루프 트랙 Gazebo — 로봇 3대

위 스케줄러·예약·복구 로직으로 Gazebo의 로봇 3대(5장)를 실제로 구동한다. 각 로봇에 VelocityControl 플러그인 + 링크 중력 off로 매달린 레일 위를 이동시키고, 디렉터가 작업 생성 → 배정 → 도착 시 예약 해제 → 재배정 사이클을 돈다. 40초간 17작업 완료.



Gazebo 페루프 — 스케줄러가 구동하는 SkyvoltRobot 3대

```
[ 0.0s] r0 -> job j0 (eta 1.4s)
[ 1.4s] r0 arrived (done 1)
[ 2.4s] r0 -> job j1 (eta 0.7s) # immediate re-assign
[ 8.8s] r2 -> job j4 (eta 2.3s) # all 3 robots active
[director] stopped. jobs completed: 17
```

9. Phase 3 — 매니플레이터 + 플러그 삽입 (핵심 IP 완료)

9.1 텔레스코픽 리드스크류 기구학

충전 플러그를 삽입하는 매니플레이터를 순수 파이썬 IP로 구현하기 시작했다. 플러그는 두 리드스크류로 구동된다 — Y축 텔레스코프가 충전구로 뻗고(320 mm), X축 래치가 체결한다(40 mm). 리드스크류는 1회전이 나사 리드만큼 전진하므로 모터각↔변위가 선형이고, 리드로 정해지는 토크→추력 이득($F = 2\pi \cdot \eta \cdot T / \text{lead}$)과 스트로크 한계를 갖는다.

- LeadScrew — 회전수↔변위, 모터각, 선속도↔rpm, 이동시간, 토크→추력, 스트로크 한계.
- TelescopicManipulator — 텔레스코프→래치 순차 삽입 계획(축별 회전수·시간), 스트로크 초과 목표 거부. 기본값이 URDF 속도 한계(0.1·0.05 m/s)와 일치.

단위 테스트 14개로 핵심 기구학을 고정 (변위 선형성·역함수, 토크→추력, 삽입 타이밍·거부). 예: plan_insertion(0.30, 0.03) → 총 3.6 s, reach 0.50 m는 스트로크 0.32 m 초과로 거부.

9.2 비전 — RGB-D 충전구 6-DoF 자세 추정

플러그를 삽입하려면 차량 충전구의 위치를 알아야 한다. RGB-D 센서가 주는 충전구 특징점(소켓 fiducial)의 3D 위치를, 충전구 자체 좌표의 모델 점과 정합해 충전구의 6-DoF 자세를 구한다.

Kabsch/Umeyama 강체 최소제곱(반사 방지 + 적합 잔차 RMSE)을 순수 파이썬+numpy로 구현했다. 추정 자세는 매니플레이터로 흘러간다 — 접근 거리는 텔레스코프 reach, 횡방향 오차는 서보가 0으로 만들 정렬 목표.

통합 데모: 0.38 mm RMSE로 충전구 검출 → 거리 280 mm·정렬오차 19.9 mm → 삽입 계획 3.4 s. 단위 테스트 9개(정확 복원·노이즈 강인성·반사 방지·매니플레이터 연결).

영상 특징 검출 프론트엔드(detector.py): Gazebo RGB-D 영상에서 어두운 소켓 4코너를 검출해 깊이·내부파라미터로 3D 역투영 → 자세 추정으로 연결. 합성 RGB-D로 영상→검출→자세→서보→도킹 전 과정을 검증(테스트 6개).

9.3 비주얼 서보잉 — 정렬 → 삽입 → 도킹 페루프

추정된 충전구 자세를 입력으로 매니플레이터를 페루프 제어한다. 2단계 제어기: ALIGN(비례 횡방향 정렬로 정렬오차를 허용치까지 축소) → INSERT(텔레스코프 전진으로 삽입) → DOCKED, 정해진 틱 안에 수렴 못 하면 FAULT. 순수 파이썬 제어기를 운동학 플랜트 대상 페루프로 검증했다.

데모: 50 mm 어긋난 충전구 → 정렬·삽입·도킹 6.8 s, 최종 정렬오차 1.96 mm (< 2 mm 허용치). 단위 테스트 9개(수렴·단조감소·위상 전진성·latch 1회·미수렴 FAULT·비전→서보 end-to-end).

9.4 힘/토크 정합 피드백

삽입 중 손목 F/T 센서의 접촉 렌치로 오정렬을 감지한다. 정렬된 삽입은 거의 순수 축방향 힘이지만, 어긋난 플러그는 소켓 림에 눌러 횡력·코킹 모멘트를 낸다. InsertionForceGuard가 렌치를 FREE/SEATING/MISALIGNED/OVERLOAD로 분류해 서보 루프에 CONTINUE/CORRECT(후퇴·재정렬)/ABORT(중단)를 권고한다.

판정 기준: 횡력/축력 비, 코킹 모멘트, 안전 한계. 단위 테스트 8개(미접촉·정상삽입·횡방향 바인딩·코킹·과부하 force/moment·우선순위).

10. Phase 4 — 운영 & 안전 (핵심 IP 완료)

운영·안전 계층(skyvolt_safety 패키지)의 소프트웨어 핵심 4건을 순수 파이썬으로 구현했다.

- 하중·기울기 모니터 — 레일 지지대 로드셀 + 경사계를 NOMINAL/IMBALANCED/OVERLOAD/TILT_FAULT로 분류해 OK/HOLD/ESTOP 권고. 구조 위험은 ESTOP 우선.

- 래칭 E-stop 체인 — 하드웨어 버튼·소프트웨어 폴트·통신 두절 감시를 묶어 래치 트립, 전 소스 해제 시에만 리셋. 하중 ESTOP이 체인 소스로 연결.
- Prometheus 매트릭 + Grafana — 플릿·안전 상태를 노출 형식으로 렌더하고 stdlib HTTP /metrics 엔드포인트로 서빙(라이브 스크레이프), 7패널 Grafana 보드 포함.
- OTA 업데이트 FSM — 다운로드→검증→스테이징→활성화, 체크섬 불일치/헬스 실패 시 롤백. 검증 버전은 헬스체크 통과 시에만 커밋(벽돌 방지).

안전 테스트 38개(하중 8·E-stop 9·메트릭 7·OTA 9·HTTP+Grafana 5). 위험 감지 → 전원 차단이 모듈 간 연결되어 검증된다.

11. 추가 / 변경된 주요 파일

파일	구분	내용
api-server/ (디렉토리)	신규	자체 완결형 REST 서버 묶음 + README/실행스크립트
skyvolt_eval/replay.py	신규	Replay 도구(trace 포맷·재생·지표·bag 변환)
skyvolt_fleet/fleet_sim.py	신규	페루프 스케줄러 스트레스 테스트 + 복구 시뮬레이션
scripts/fleet_director.py	신규	Gazebo 3대 로봇 페루프 디렉터
skyvolt_manipulator/ (패키지)	신규	리드스크류 기구학 + 힘/토크 가드 + 22 테스트
skyvolt_vision/ (패키지)	신규	자세 추정 + 서보잉 + RGB-D 특징 검출 + 24 테스트
skyvolt_safety/ (패키지)	신규	하중·E-stop·메트릭(HTTP/Grafana)·OTA + 38 테스트
reservation.py	수정	reclaim_expired() 타임아웃 회수(데드락 복구)
api/server.py	수정	운영자 대시보드(GET /) + /fleet 트랙길이 추가
scripts/*.sh	신규	테스트·도킹평가·빌드·시뮬레이션 실행 스크립트
test_*_properties / replay / fleet_sim	신규	속성·Replay·플릿 스트레스 테스트
track_localizer.py	수정	IMU 가속도 제어입력 + 자이로 곡선 속도 융합
localizer_node.py	수정	/imu 구독 추가(sensor_msgs)
urdf / launch 파일	수정	관성·Ogre v1·VelocityControl·중력 off
docs/ROADMAP.md	수정	Phase 1·2 완료, Phase 3 기구학 착수 체크

12. 실행 방법

```
cd skyvolt_mvp
./scripts/run_tests.sh          # all tests (149)
./scripts/run_docking_eval.sh   # docking eval x100
python -m skyvolt_eval.replay --trace t.jsonl # replay -> metrics
python -m skyvolt_fleet.fleet_sim --jobs 1000 --robots 10 # stress test
./scripts/build_ros.sh          # build ROS workspace (first time)
./scripts/run_sim.sh fleet      # Gazebo 3-robot fleet (terminal 1)
python3 scripts/fleet_director.py # closed-loop director (terminal 2)
cd api-server && ./run.sh        # dashboard + API (8080)
```

13. 다음 단계

Phase 0~4의 순수 소프트웨어 핵심 IP가 모두 구현·검증됐다(138 테스트). 남은 항목은 전부 하드웨어/전송/sim 연동이다:

- Phase 1 #1 — 실측 RFID 노이즈 모델: 실제 RFID 리더 장비 필요. [HW]
- Phase 3 — 접촉 물리 sim(F/T 실데이터), 실제 카메라 영상 프레임링·검출 튜닝. [sim]
- Phase 4 — HW GPIO(E - stop) 결선, OTA 전송 계층, Prometheus·Grafana 배포. [HW/인프라]